3주차

DF 2026 가을 단기 프로그램

# 사이킷런과 전처리

파이썬 머신러닝 완벽 가이드 P85–144

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
```

# 목차


01 사이킷런 기본 — 패키지 구조 · 용어 · 분류 프로세스 · Estimator

02 검증과 튜닝 — 데이터 분리 · 과적합 · 교차 검증 · 하이퍼파라미터 튜닝

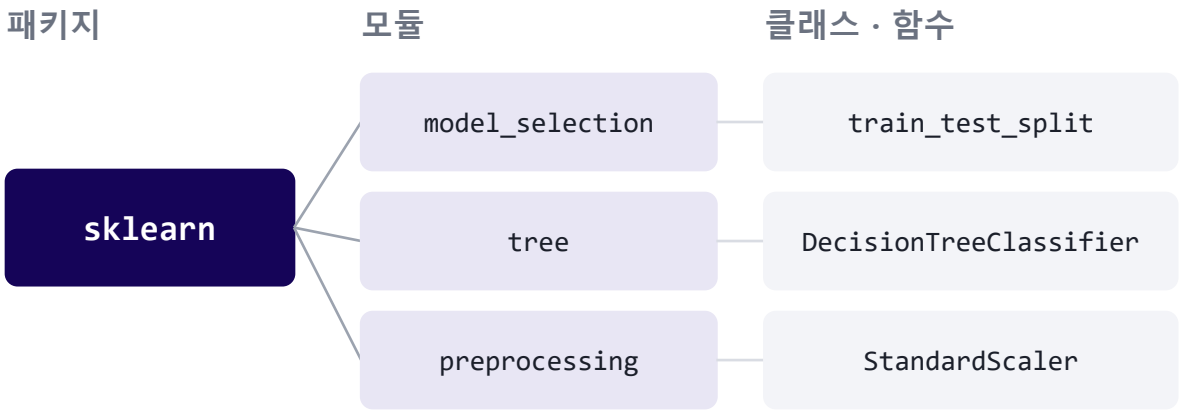
03 데이터 전처리 — 입력 조건 · 인코딩 · 피처 스케일링

04 3주차 프로젝트 — 분류 · 회귀 데이터 각각 예측 수행

# 사이킷런과 파이썬 계층 구조



패키지 → 모듈 → 클래스 · 함수 · 변수




**라이브러리**
특정 목적을 위한 패키지 · 모듈 모음

**import**
다른 곳에 정의된 패키지 · 모듈 · 클래스 · 함수를 가져와 사용

```
from sklearn.model_selection import train_test_split   # 함수
from sklearn.tree import DecisionTreeClassifier        # 클래스
```

# 용어 정리

붓꽃(iris) 데이터 기준

| sepal length | sepal width | petal length | petal width | label |
|---|---|---|---|---|
| 5.1 | 3.5 | 1.4 | 0.2 | setosa |
| 4.9 | 3.0 | 1.4 | 0.2 | setosa |
| 7.0 | 3.2 | 4.7 | 1.4 | versicolor |
| 6.4 | 3.2 | 4.5 | 1.5 | versicolor |
| 6.3 | 3.3 | 6.0 | 2.5 | virginica |

petal 꽃잎 · sepal 꽃받침 · length 길이 · width 너비

**피처 (feature)**
모델 입력 변수 = X

**레이블 (label)**
정답 클래스 · 예측 대상 = Y

**파라미터 (parameter)**
학습으로 정해지는 값 (예: 회귀 계수 W)

**하이퍼파라미터**
학습 전 사람이 지정하는 값 (예: max_depth)

# 붓꽃 품종 분류 프로세스



학습 데이터로 모델 학습 → 테스트 데이터 예측 → 실제값과 비교

## 학습 데이터

| 꽃받침 길이 | 꽃받침 너비 | 꽃잎 길이 | 꽃잎 너비 | 품종 |
|---|---|---|---|---|
| 5.1 | 3.5 | 1.4 | 0.2 | setosa |
| 7.0 | 3.2 | 4.7 | 1.4 | versicolor |
| 6.3 | 3.3 | 6.0 | 2.5 | virginica |

→ **모델 학습**
`fit()`

↓ 학습된 모델

## 테스트 데이터

| 꽃받침 길이 | 꽃받침 너비 | 꽃잎 길이 | 꽃잎 너비 | 품종 |
|---|---|---|---|---|
| 4.9 | 3.0 | 1.4 | 0.2 | ? |
| 6.4 | 3.2 | 4.5 | 1.5 | ? |

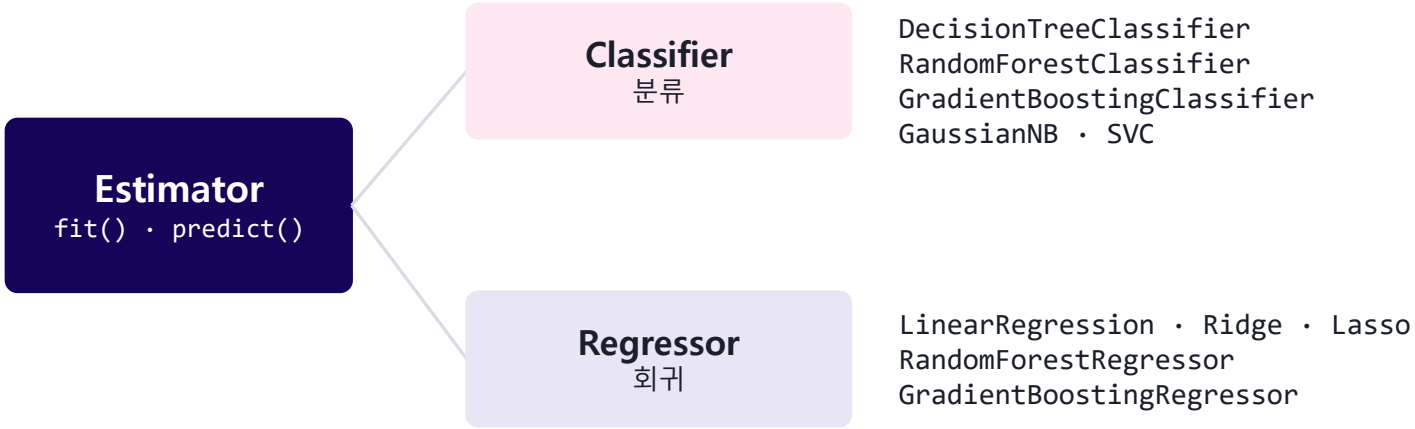
→ **예측**
`predict()`

→ **평가**
예측값 vs 실제값

예: 결정 트리로 붓꽃 30개 예측 → 정확도 약 93%

# Estimator 구조

학습 fit() · 예측 predict()가 공통으로 내장




**전처리 · 비지도 학습 클래스** fit() 기준 정보 학습 → transform() 실제 변환, 한 번에 fit_transform()

# 데이터 분리

학습에 쓴 데이터로 평가하면 성능이 부풀려짐 → 분리 후 평가




**학습 데이터**
모델이 패턴을 배우는 데이터

**검증 데이터**
하이퍼파라미터 튜닝 · 모델 선택

**테스트 데이터**
최종 성능 평가, 마지막에 한 번만 사용

## train_test_split() 주요 인자

**`test_size`**
테스트 데이터 비율 (예: 0.2)

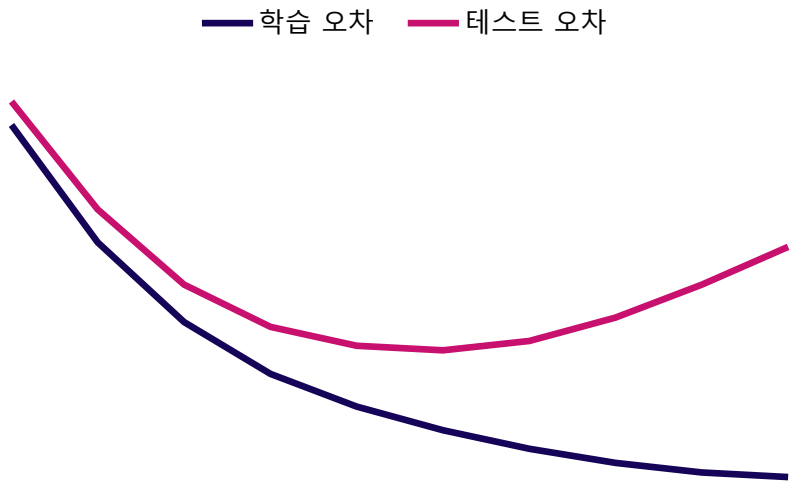
**`random_state`**
같은 결과를 재현하기 위한 고정값

**`stratify`**
레이블 비율을 학습 · 테스트에 동일하게 유지

# 과적합 (Overfitting)



학습 데이터에만 지나치게 맞춰져 새 데이터에서 성능이 떨어지는 현상




**과소적합**
학습 · 테스트 오차 모두 큼, 모델이 너무 단순

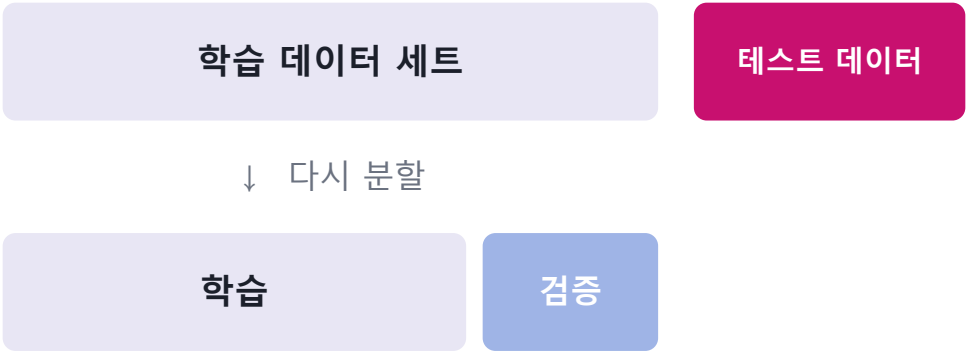
**적정**
테스트 오차가 가장 낮은 지점에서 멈춤

**과적합**
학습 오차는 계속 감소, 테스트 오차는 증가 → 학습 데이터를 외운 상태

MSE (평균 제곱 오차): (실제값 − 예측값)²의 평균

# 교차 검증 (Cross Validation)

모델이 얼마나 잘 일반화되었는지 확인하는 방법




테스트 데이터는 모든 학습 · 검증이 끝난 뒤 최종 평가에만 사용

**왜 필요한가**
한 번의 분할 결과에 성능이 좌우되는 문제 → 여러 번 나눠 평균

**목적**
과적합 방지, 처음 보는 데이터에서도 잘 맞는지(일반화) 확인

**대표 방법**
K-fold · Stratified K-fold

# K-fold 교차 검증

K번 학습 · 검증 후 평균으로 최종 평가 (K = 5 예시)




1. 데이터를 K개 폴드로 분할
2. K−1개로 학습, 1개로 검증
3. 검증 폴드를 바꿔 가며 K번 반복
4. K개 점수의 평균으로 성능 판단

**최종 평가 = 평균(평가 1~5)**

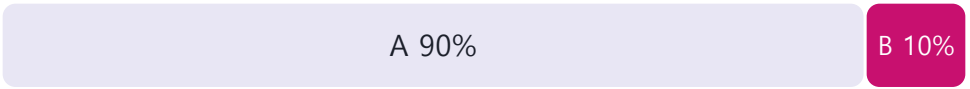
# Stratified K-fold

원본 데이터의 레이블 분포를 각 폴드에 그대로 반영

원본 데이터




일반 K-fold의 한 폴드




**Stratified K-fold의 한 폴드**




(비율은 예시)

**언제 쓰나**
레이블이 한쪽으로 치우친 분류 데이터 (예: 사기 거래, 질병 여부)

**일반 K-fold의 문제**
어떤 폴드에는 소수 클래스가 거의 없어 평가가 왜곡

**회귀에는**
레이블이 연속형이라 사용 불가 → 일반 K-fold

# cross_val_score()

교차 검증 전 과정을 한 번에 수행하는 API




**분류**
내부적으로 Stratified K-fold 사용

**회귀**
일반 K-fold 사용

**cross_validate()**
여러 평가 지표를 한 번에 반환

**시계열 데이터**
순서가 섞이므로 일반 교차 검증 부적합

# GridSearchCV

교차 검증 + 하이퍼파라미터 튜닝을 한 번에

| max_depth ↓ \ min_samples_split → | 2 | 3 |
|---|---|---|
| 1 | CV 3회 평균 0.70 | CV 3회 평균 0.70 |
| 2 | CV 3회 평균 0.93 | CV 3회 평균 0.94 |
| 3 | CV 3회 **평균 0.97** | CV 3회 평균 0.95 |

6개 조합 × 3폴드 = 18번 학습 · 평가 (점수는 예시)

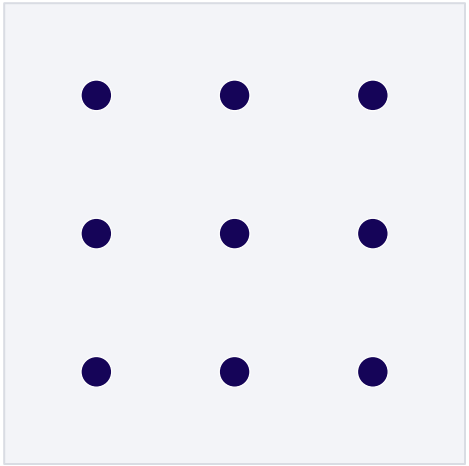
1. 하이퍼파라미터 후보 값 지정
2. 모든 조합을 교차 검증
3. 평균 점수가 가장 높은 조합 선택 (best_params_)
4. refit=True → 그 조합으로 학습 데이터 전체 재학습 (best_estimator_)

**튜닝은 학습 데이터에서만**
테스트 데이터는 최종 평가용으로 보존

# Grid vs Randomized Search

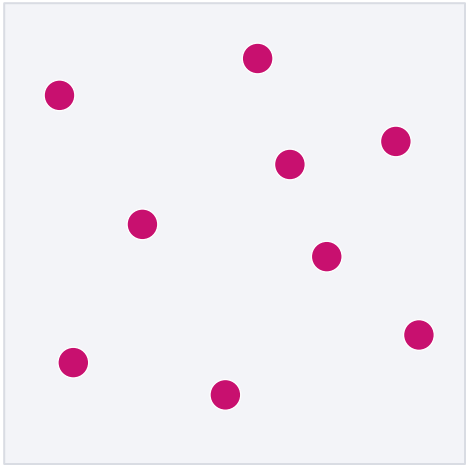
모든 조합 탐색 vs 정해진 횟수만큼 무작위 탐색

**GridSearchCV**



파라미터 A →

**RandomizedSearchCV**



파라미터 A →

**GridSearchCV**

지정한 값의 모든 조합 검증

파라미터 간 상호작용이 클 때

**RandomizedSearchCV**

n_iter 횟수만큼 무작위 조합 검증

탐색 공간이 크고 파라미터 간 상관이 낮을 때

# 사이킷런 입력 조건

결손값 없이, 숫자로만

## 결손값 (NaN)

- 판다스는 NaN · None 모두 결손값으로 인식
- 대부분의 사이킷런 모델은 결손값 입력 시 오류
- 처리: 평균 · 중앙값 · 최빈값 대체 또는 삭제

`df.isna().sum()` 으로 먼저 확인

## 문자열

- 모델 입력은 숫자형만 가능
- '01', '02' 같은 코드값도 문자열이면 변환 필요
- 처리: 인코딩 (다음 장)

```
ValueError: could not convert string to float:
'female'
```

# 범주형 인코딩 세 가지

같은 Color 칼럼을 서로 다른 방식으로 변환

## 레이블 인코딩

`LabelEncoder`

| Color | 인코딩 |
|---|---|
| Red | 2 |
| Green | 1 |
| Blue | 0 |

숫자 크기가 순서 · 가중치로 오해될 수 있음 → 선형 모델 부적합, 레이블(Y) · 트리 계열에 사용

## 원-핫 인코딩

`pd.get_dummies(df)`

| Blue | Green | Red |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |

범주 수만큼 칼럼 생성 → 범주가 많으면 피처 수 급증

## 더미 인코딩

`drop_first=True`

| Green | Red |
|---|---|
| 0 | 1 |
| 1 | 0 |
| 0 | 0 |

첫 범주 칼럼 제거 → 칼럼 1개 감소, 모두 0이면 Blue

입력 순서: Red, Green, Blue · LabelEncoder는 알파벳순으로 번호 부여 (Blue 0, Green 1, Red 2)

# 피처 스케일링

서로 다른 변수의 값 범위를 일정한 수준으로 맞추는 작업

## 거리 계산 예시

| | 나이 | 연봉(만 원) |
|---|---|---|
| A | 25 | 3,000 |
| B | 55 | 3,100 |
| **차이** | **30** | **100** |

나이 차이가 더 의미 있어도 숫자가 큰 연봉 차이가 거리를 좌우 → 스케일을 맞춰야 공정한 비교 가능

### 필요한 모델

- 선형 회귀 + 규제 (릿지 · 라쏘)
- KNN · SVM 등 거리 기반
- PCA 등 분산 기반 차원 축소

### 필수 아닌 모델

- 결정 트리 · 랜덤 포레스트 등 트리 기반
- 값의 크기가 아닌 분할 기준으로 학습

# 스케일링 적용 규칙

기준은 학습 데이터에서만 만들고, 테스트 데이터에는 그대로 적용




**테스트 데이터에 다시 fit() 금지**
기준이 달라져 학습 · 테스트 데이터가 서로 다른 척도로 변환됨

**변환 결과는 ndarray**
칼럼명이 사라지므로 필요하면 DataFrame으로 다시 변환

# 스케일러 비교

정규화 · 표준화 · 로버스트 스케일링

## StandardScaler

표준화

$$(x - 평균) / 표준편차$$

**결과**
평균 0, 표준편차 1 (범위 제한 없음)

**적합한 경우**
피처가 정규분포에 가까울 때, 단위가 제각각인 연속형 피처

## MinMaxScaler

정규화

$$(x - 최소) / (최대 - 최소)$$

**결과**
0 ~ 1 범위

**적합한 경우**
분포를 모를 때, 이상치에 매우 취약 (최대 · 최소가 기준)

## RobustScaler

로버스트

$$(x - 중앙값) / IQR$$

**결과**
평균 · 범위 보장 없음

**적합한 경우**
이상치가 많을 때 (중앙값 · 사분위수 기준이라 영향 적음)

# 정규화 vs 표준화

둘 다 스케일을 맞추지만 기준과 결과가 다름

| | 정규화 (Normalization) | 표준화 (Standardization) |
|---|---|---|
| **사용하는 값** | 최솟값 · 최댓값 | 평균 · 표준편차 |
| **결과 범위** | 0 ~ 1 | 제한 없음 (평균 0, 표준편차 1) |
| **유용한 경우** | 분포를 모를 때, 피처 크기가 서로 다를 때 | 피처가 정규분포에 가까울 때 |
| **이상치** | 매우 취약 | 영향 있음 → 이상치가 많으면 RobustScaler |
| **사이킷런 클래스** | MinMaxScaler | StandardScaler |

# 프로젝트 ① 분류 데이터

모든 조가 분류 · 회귀 두 가지 모두 진행

1. **범주형 변수 인코딩** LabelEncoding · One-Hot Encoding
2. **X, Y 변수를 새 DataFrame으로 추출**
3. **train_test_split으로 데이터 분리**
4. **fit() · predict()로 예측 수행** Estimator: DecisionTree · RandomForest 중 선택
5. **교차 검증 후 정확도 비교** KFold · Stratified KFold
6. **GridSearchCV로 정확도 비교 또는 최적 파라미터 탐색**
7. **파라미터 선택 이유 PPT에 작성**

**제출** PPT 정리 후 노션 A반 과제 3주차에 업로드

# 프로젝트 ② 회귀 데이터

분류와 같은 흐름, 인코딩 대신 스케일링에 집중

1. **적절한 피처에 스케일링** 정규화 · 표준화에 적합한 피처 확인 후 진행
2. **X, Y 변수를 새 DataFrame으로 추출**
3. **train_test_split으로 데이터 분리**
4. **fit() · predict()로 예측 수행** Estimator: LinearRegression
5. **교차 검증 후 성능 비교** KFold, 지표는 RMSE · $R^2$ 등
6. **GridSearchCV로 성능 비교 또는 최적 파라미터 탐색**
7. **파라미터 선택 이유 + 스케일링 적용 피처와 이유 PPT에 작성**

**제출** PPT 정리 후 노션 A반 과제 3주차에 업로드

# 조별 PPT 구성 예시

발표 자료 구성 순서

**01**

**전처리 요약**

인코딩 · 스케일링 대상과 방법

**02**

**분리 · 모델**

test_size, 사용한 Estimator

**03**

**교차 검증 결과**

폴드별 점수와 평균

**04**

**튜닝 결과**

best_params_, 튜닝 전후 점수

**05**

**선택 이유**

파라미터 · 스케일링 선택 근거

**분류 · 회귀 각각 점수 비교표 하나씩: 기본 모델 → 교차 검증 평균 → 튜닝 후**